CS 1050 Technical Documentation

Click table of contents below to refresh after adding more headings

Overview

When you add information in your own words demonstrating your understanding. Do not use exact words from lectures or images from lectures.

DOs and DON'Ts
DO: Provide clear and simple explanations in your own words (be concise) • Use short sentences, small paragraphs • Use lists and tables to organize information • Use standard industry terms but explain in simple language • Use AI Tools to help you learn but use lectures first **DON'T copy my explanations from the slides as yours.**
DO Use Code Snippets and Visuals to reduce lengthy explanations: • Include examples you explored or created • Include comments to describe parts of code • Include Screenshots such as from your debugger

- Include your own drawings such as for memory, arrays, etc.
- Include images you have AI create (cite that AI created)
- Include images from the Internet with the link

DON'T use my slides as your image and explanations

DO Include Resources that give steps or help explain concepts

- You can include resources given in the lecture or find your own but
  - Include website link with summary
  - Include links to videos with summary of content

DON't use AI to create your technical documentation.

General Resources

- Your first source should always be your course lecture slides and documents or links to information in the lectures
  - F26_CS1050_Student_Course_Resources
  - Java Environment Setup
  - Introduction to Programming with Java - use the sections shared with you to practice ideas from the lecture.
- Next resource is to see Deb or learning assistance during office hours
- If you're unsure whether a source is trustworthy, ask your instructor first.

  ⚠️ Avoid These Sources
  - TPointTech https://www.tpointtech.com/
  - W3Schools – Java, syntax, data types, etc.: https://www.w3schools.com/java/
  - Sites without a known author or date
  - AI-generated blogs that do not cite documentation
  - Use of AI should be last as it will probably give you information you haven't learned or understand. It is expected you will be documenting concepts from class and book using your code examples.

Other Tools you should use:

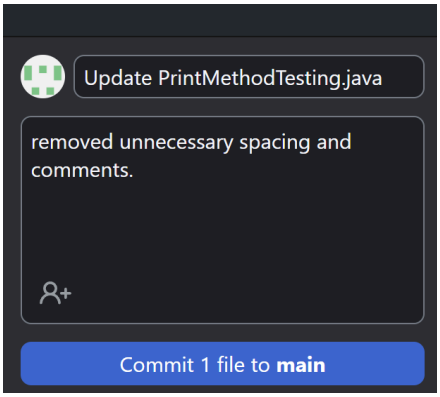
- Eclipse
- Visualize Java Code - used to step through code and edit code

SD01 Software Development and Tools

Github	Git
Store your code and create versions	Command line tool.

of it. Web service.	MANAGES versions of code. Typically already pre-installed on the computer

Version Control



Commiting

When you commit, you are adding versions of your code to your branch/history.

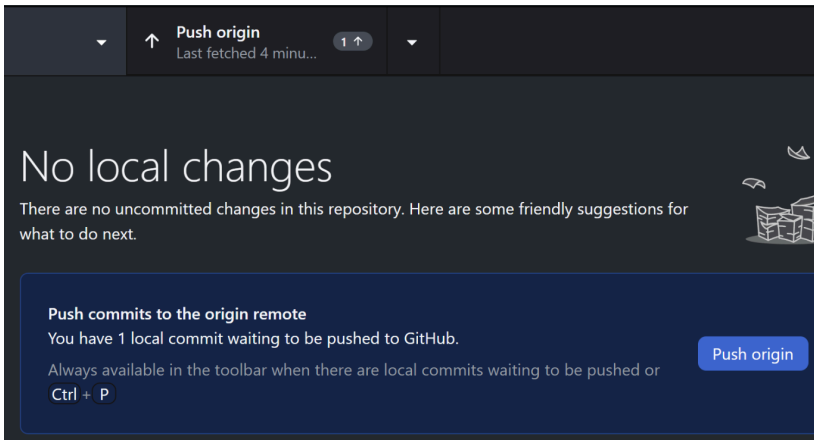
You can describe things before you commit them in order for it to be easier to find versions in your history!

In your history, all your old commits are displayed! The description is in the middle near the top of the screen.

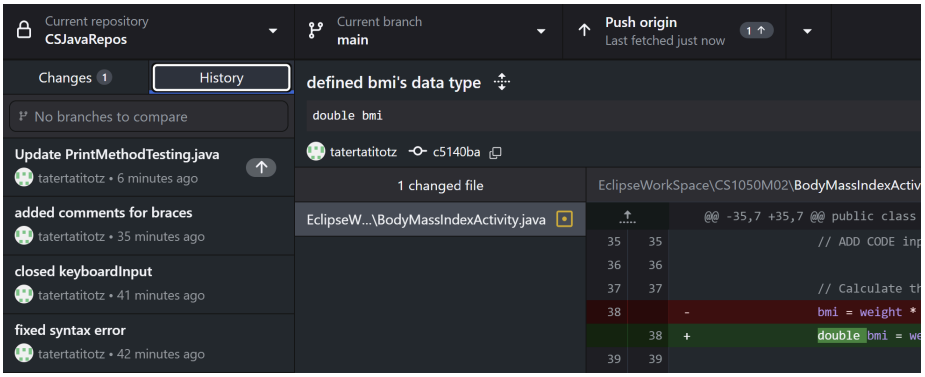
Pushing

Pushing can be done in two places, at the top, and when there are **NO CHANGES**, they are in the window that displays your code.

Push origin button is the most consistently accessible button.



History



History can be found on the top right of Desktop.

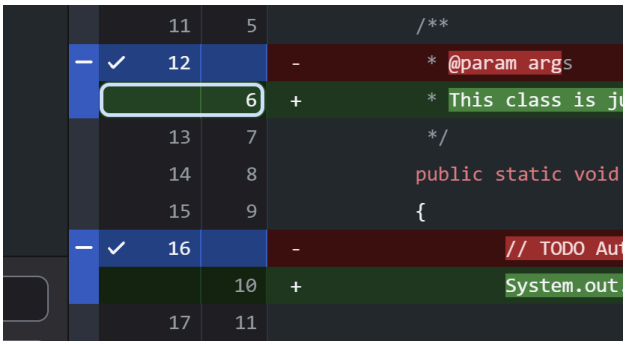
On the right shows all your changes.

At the top shows your Summary and description.

Versioning

Looking back on your history is crucial to versioning! It displays all versions of your code (that you saved) for you to look back on.

Selection Indicators

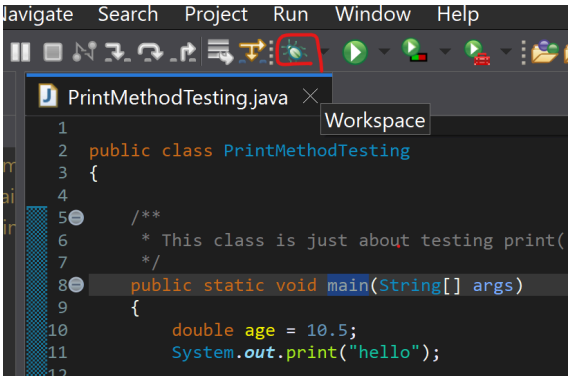


You are able to be selective with your changes.

The blue checkmark indicates which ones have been selected to commit.

If it has a blue line then that means it has only been partially selected.

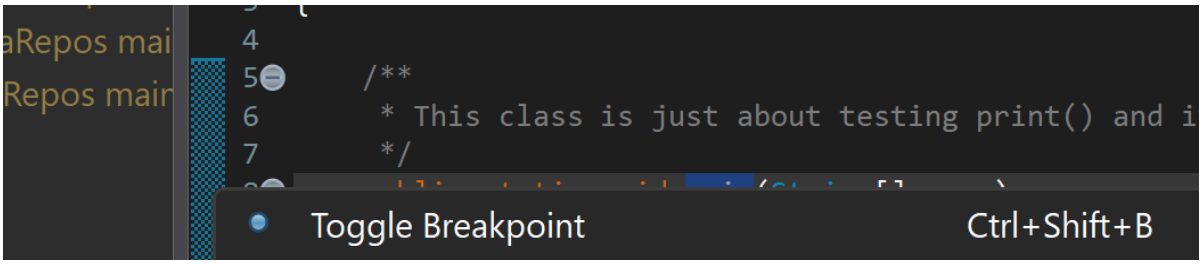
IDE and Debugging



The Debug is at the top of the screen under the bar with all the words on it.

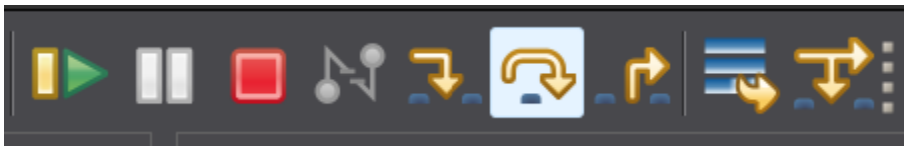
It opens a menu on the right with all variables as you go through each step.

Don't forget to add break points before pressing it by right clicking on the blue mesh on the left of the editor:



Debug Navigation

The highlighted button is to go step by step. The angular one to the left of it, is going into the much deeper parts of the code.



Analyze Requirements

Analyze requirements(user stories and acceptance criteria) to break problems into smaller testable tasks

Design and Testing Plan

- Apply Test-Driven Development (TDD): Create unit test cases, design algorithms in pseudocode, implement code (structured, readable and documented), debug and unit test incrementally
- Develop a documented plan by creating unit test cases and algorithm designs
- Use UML to design and implement multiple class programs.

Implement Quality Documented Code

Java Naming Conventions

Variables	Constants	Methods	Classes
camelCase	SCREAMING_SNAKE_CASE	camelCase()	PascalCase

- Write comments at top of file and inline comments to document code.
- Use constants and variables and avoid hard coding literals
- Document method code following javadoc commenting
- Use Single Responsibility Principle (SRP) and Do Not Repeat Yourself (DRY) to implement modular, readable and testable solutions

Module 1: Programming Fundamentals and Tools

Add the information to demonstrate the following objectives and organize with headings:

Declaring, Initializing, and Assigning

Declaring	Initializing	Assigning

Example:
`//creating space`
`double variable;`
`int variableAgain;`
Basically making space to put these variables.
	Examples:
	`//setting initial value`
	`double variable = 20.3;`
	`int variableAgain = 10;`
	`//DECLARE FIRST`
	`variable = 20.3;`
	`variableAgain = 10;`
	You assign FOR THE FIRST TIME a variable to the identifier.
	
	In the second example, they must be declared ahead of time.
		Example:
		`double variable = 20.3;`
		`int variableAgain = 10;`
		`/*Assigned after */initializing`
		`double variable = 10.1;`
		`int variableAgain = 15;`
		Assigning after the first variable has been initialized is called, well… assigning.

Constants & Variables

Constants	Variables

`//final means it is a constant`
`//int is a single whole number`
`//ALWAYS… Is the identifier`
`final int ALWAYS_CAPITALIZED = 400;`
Constants are constant. They never change and they are cased and labelled very specifically so they can be efficiently identified.
	`double variable = 20.3;`
	`int variableAgain = 10;`
	
	`double variable = 10.1;`
	`int variableAgain = 15;`
	Variables are initialized and then assigned later. They CHANGE.

Order of Operations

//Parenthesis, Exponents, Multiplication, Division, Addition, Subtraction is the order of operations.

Integer Division (/)	Modulus Operator (%)
Integers being divided do NOT give decimal points. RED IS EXCLUDED 3 / 2 = 1.5 1 / 2 = 0.5 Anything after the decimal point is removed when dividing integers. TRUNCATED	Think long division. What is the remainder if these numbers were divided? 12 % 5 = 2 You can only fit 5 into 12 twice, leaving a remainder of 2.

Memory Allocation Based on Data Type

Data Types	Byte	Short	Int	Long	Float	Double
Size	1 byte	2 bytes	4 bytes	8 bytes	4 bytes	8 bytes

Overflow Errors During Program Execution

Overflow	

//4 byte
int integer = 12;
//8 byte
double double = 11.2;
//8 bytes DOESN'T FIT in 4 bytes
integer = double
	//2 byte
	short short = 12;
	//8 byte
	long long = 11.2;
	//8 bytes DOESN'T FIT in 2 bytes
	short = long

OVERFLOW ERROR	OVERFLOW ERROR

Ints and Doubles

INT

```
/*
* Whole numbers only
* Good for simple arithmetic
*/

int haveThisInteger = 4;
```

1

DOUBLE

```
/*
* Numbers with decimal points
* good for doing precise division
* percent calculation
*/
double randomDouble = Math.random();
```

1.1

Arithmetic Operators

Symbol	+	-	*	/	%
Input	1 + 2	2 - 1	5 * 3	4 / 2	7 / 3
Result	3	1	15	2	1

Implicit and Explicit Conversion Rules and Casting

Type Casting

Explicit Casting

```
int eggs = 12;
double flourCups = 3.5;
/*(int) tells the computer that
* flourCups is now an int
* Turning a double into an int truncates
* it. (Removes decimal)
*/
int eggs = (int) flourCups;

System.out.println(eggs);
```

Console:

3

Numeric Conversions

```
int eggs = 12;
double flourCups = 3.5;

double recipe = eggs * flourCups;

System.out.println("recipe =" + recipe)
```

Console:

recipe = 42

Mixed Data Type Operations

// int / int (3 / 2) = 1	// double / int (3 / 2.0) = 1.5	// double / double (3.0 / 2.0) = 1.5

```
/*
* Mixed Data Types are like OR Gates. As long as one of the values is a double,
* the result will be the same
*/
```

Standard Output System.out.print[ln, f]()

System	out	print[ln]()
// retrieves methods	// display console	// method

print()	println()

// prints out strings in console
System.out.print("Hello!")
 	
// Console below:
	System.out.print("Hello! ")
	// makes a new line after printed
	System.out.println("My name is:")
	// is placed on new line after ln
	System.out.print("I forgot.")

Console output:	
Hello!	Hello! My name is: I forgot.

printf()

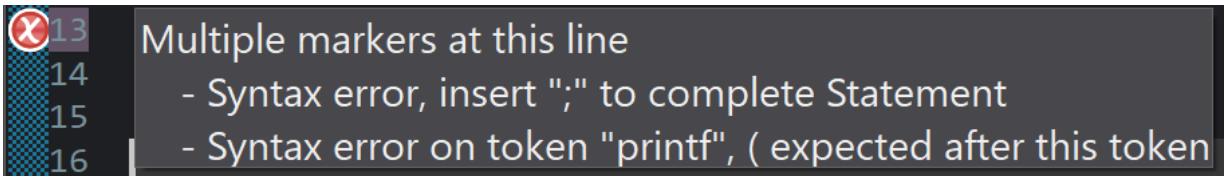
• %b • %c • %d • %f	• Boolean • Character • Integer • Double	char charTest= 'a'; int charAsciiValue= (int)charTest; String firstName =

• %e • %s	• Sci Notation • String	"Heriberto"; System.*out*.printf("char: %c ascii value: %d \n", charTest, charAsciiValue);

Types of Errors

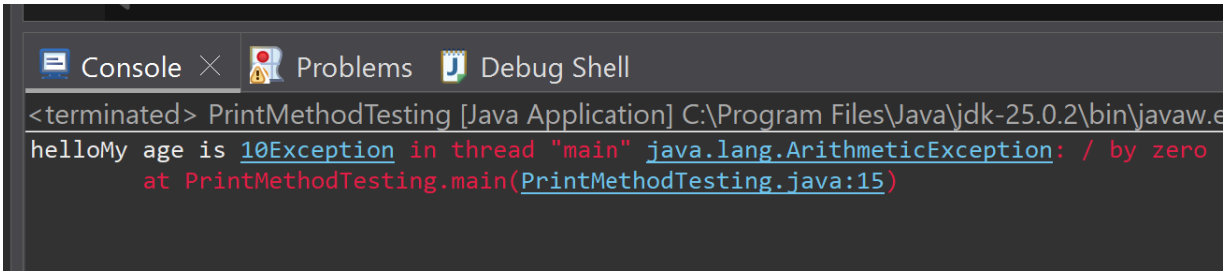
Syntax Errors

Syntax errors are there when something like a parenthesis, curly brace, or semi colon are missing. It's a lot like when you forget to place a period or a comma and the whole sentence collapses.



Runtime Errors

Runtime errors occur when something happens in the code ONLY when it is run. There are no error messages for Runtime errors. You know you have one when your console is telling you something is wrong.



Logical Errors

Logical Errors happen when your expected result isn't the result you actually get. This is the humans' fault. For example, if you want to multiply 6 and 9 before putting it to the power of 2, you need parentheses or else you'll get the wrong answer.

Module 2: Predefined Classes, Methods, and Decision Structures

Using built-in classes (System, Scanner, String, Character, Math) and their methods; parameters and return values

Classes are non-primative data types.

System

System Methods

```
System.out.println("This program will calculate your " + "body mass index, or BMI.");
```

Before deciding a method, you must indicate where the method is occurring. Out means it is being put in the console.

Return Values

Value returned in a string in the console that says “This program will calculate your body mass index, or BMI.”

Scanner

Scanner Methods

```
String name1 = keyboardInput.next();
weight = keyboardInput.nextDouble();
```

The scanner is named keyboardInput and the methods read the next thing coming from the scanner.

Return Values

Name1 = whatever you typed as a STRING
Weight = whatever NUMBER, INCLUDING decimal, you typed.

String

String Methods

```
String firstName = "String";
// measures the # of chars in a String
int aLengthInt= firstName.length();
// points at individual chars at an assigned
char aChar= firstName.charAt(3);
```

Return Values

```
Console:
length of the String: 9
fourth letter: i
```

Comparing Strings:
`USERNAME_ACTUAL.equals(inputedUsername)`
`USERNAME_ACTUAL.equalsignoreCase(inputedUsername)`
	The return value is a boolean, yes or no. It is just checking if it is true or false

Character	
Character Methods `section = Character.toUpperCase(keyboardInput.next().charAt(0));`	Return Values The first char of a String you just typed. No Object
Math	

String Math
`// You can cast the results`
`double randomDouble = Math.random();`
	Return Values
	Random number:
	0 =< x < 1
	
	No Object

Distinguish Characters (char) and Strings

char	String
Chars use ‘ ‘ instead of “ “  Chars are assigned a number value from the Ascii table. Chars are concatenated into Strings.	“ ”  A string is a series of chars all strung up.
Primitive	Non-Primative

String as an array of characters and String class methods.

String as Array	String Class Methods

`char lastInitial = keyboardInput.next().charAt(1);`	`char lastInitial =`
	`// next() sees strings until return or space`
	`// charAt(0) sees one of the characters in a string`
	`keyboardInput.next().charAt(0);`

You can choose from where in the array you are taking characters from. Each Character is assigned a number starting from 0.	next() reads a string until the next space charAt() reads a character in the string depending on the space.

Relational Operators			Boolean Values
<	<	Less than	Just testing these things are true and false. (3 > 1) = True (4 = 3) = False Just use your head for these. YOU CANNOT CHAIN LOGICAL AND RELATIONAL OPERATORS SUCH AS: Fruit != 2 || 3 || 4 || 5 They have to be applied like this: Fruit ! = 2 || Fruit ! = 3 || Fruit ! = 4 etc… Putting them in parentheses changes nothing. In this case, doing a multiway-if or a switch is more ideal than typing them all out if you want separate outputs for each one.
<=	≤	Less than or Equal to	
>	>	Greater than	
>=	≥	Greater than or Equal to	
==	=	Equal to	
!=	≠	NOT Equal to	

If-Else, Multi-Way If, and Nested If Statements

If-Else

```
if (bmi >= 40)
{
        System.out.println("Obese");
}
else
{
        System.out.println("Underweight");
} // end else
```

If it is conditional.

Else is IF the initial condition isn't met.

Multi-way If

```
// multi-way if
                if (bmi >= 40)
                {
                        System.out.println("Obese");
                } else if (bmi >= 25)
                {

System.out.println("Overweight");
                } else if (bmi >= 18.5)
                {
                        System.out.println("Normal");
                } else
                {

System.out.println("Underweight");
                } // end else
```

An If statement has multiple parts to it using else if.

Great for things that fall in a multitude of ranges.

Nested If

```
if (haveThisInteger > 3)
{
                if (haveThisInteger == 4)
                {
                System.out.print("Integer = 4");
                }
}
```

An If statement within an If statement.

Example:

If something is greater than 3, but you want a more specific number to print something, that is a nested if.

Scope of Variables

```
if (number1 < number2)
{
    //declared and initialized
    int temp = number1;
    number1 = number2;
    number2 = temp;
    System.out.println("temp value is " + temp);
}
System.out.println("temp value is " + temp);
```

If temp is declared inside braces, it cannot be called if the call is coming from outside the braces.

It's like trying to pick up a phone that's ringing in a different room than you.

Logical Operators

||	or	If one boolean is true then output True
&&	and	If both booleans are true then output True
!	not	!= Not Equal,

Switch Statements

```
switch (number)
{
        case 1:
        {
        System.out.print("This sure is a 1");
        break;
        }
        case 2:
        {
        System.out.print("2 for you");
        break;
        }
        case 3:
        {
        System.out.print("3 for me");
        break;
        }
        default:
        {
        System.out.print("What is this <:'O");
        break;
        }
}
```

Switch is the beginning of the switch statement. One of the parameters is a specified variable called an expression.

Cases are values tied to the expression. They must be the same data type as the expression

Break stops execution.

If none of the cases are met, the default cause executes.

You CANNOT use Relational or Logical Operators.

Works amazing with discrete sets of data.

Module 3: Loops and Methods

Add the information to demonstrate the following objectives and organize with headings:

Increments

Prefix

```
int prefix = 5;
        System.out.println(prefix);
        System.out.println(++prefix);
```

Console:

5
6

Changes the one before it

Postfix

```
int prefix = 5;
        System.out.println(prefix++);
        System.out.println(prefix);
```

Console:

5
6

Changes the one after it

Decrements

Prefix

```
int prefix = 5;
        System.out.println(prefix);
        System.out.println(--prefix);
```

Console:

5
4

Changes the one before it

Postfix

```
int prefix = 5;
        System.out.println(prefix--);
        System.out.println(prefix);
```

Console:

5
4

Changes the one after it

While

```
//while exponent negative or zero
while (exponent <= 0)
        {
//while exponent negative or zero
        System.out.println("Error: Exponent inputted
```

Do While

```
do
{
        // Prompt for input
        System.out.print("Enter a positive number for
an exponent: ");
```

```
is negative or zero.");
            System.out.print("Enter a positive number for
an exponent: ");
            exponent = keyboardInput.nextInt();
            }
```

A while loop is while something is meeting the parameter, the contents within it will continue to execute.

```
            // initializes exponent
            exponent = keyboardInput.nextInt();
            if (exponent <= 0);
            {

            System.out.println("Error: Exponent inputted
is negative or zero.");
            }
} while (exponent <= 0);
```

Do while, is that the while will always execute once. As long as the parameter of while is met it will continue looping.

For

```
/*
 * count begins at 1, because the exponent CANNOT be 0. As long as count is less
 * than the exponent, every time it loops, it will count up.
 */
for (int count = 1; count < exponent; ++count)
            {
                        // will multiply the base number by its original value until count is equal to
                        // exponent
                        base = base * baseMult;
            }
```

Sentinel Values	Boolean Flags
`while (exponent <= 0)` 0 is the parameter in which the loop stops. It doesn't have to be an int, it can just have a readable value.	`boolean loggedIn = false;` Boolean flags are used to track whether a condition has been met or not in control structures. Think of it like a light switch.

Scope

Control Structure

```
for (int attempts = 0; attempts <= 3; ++attempts)
        {
        boolean loggedIn = false;
        }
```

The integer attempt is within the parameters of the for loop, meaning it is ONLY within the scope of the for loop.

The boolean loggedIn is also only within the scope of the for loop, because it is declared within the BRACES.

Method

```
public static void main(String[] args)
{
Scanner keyboardInput = new Scanner(System.in);
System.out.print("What is your max?: ");
int max = keyboardInput.nextInt();
}// end of userMax

// max is now defined as inputtedMax
public static int checkMax(int inputtedMax)
{
        while (inputtedMax <= 0)
        {
        Scanner keyboardInput = new Scanner(System.in);
        max = keyboardInput.nextInt();
        keyboardInput.close();
        }
return inputtedMax;
}// end of checkMax
```

All variables and datatypes must be passed to the method before use via the parameter.

Max is declared outside of the checkMax method, meaning that it is not in the SCOPE of the new method.

keyboardInput is also declared outside of checkMax, so the Scanner must be built AGAIN.

Nested Control Structures/Loops

```
do
{
// the number of attempts begins at 0, there are no more than 3 attempts allowed
        for (int attempts = 0; attempts <= 3; ++attempts)
        {
        System.out.print("Input username: ");
        inputedUsername = keyboardInput.next();
        // initialized inputedPassword
        System.out.print("Input password: ");
        inputedPassword = keyboardInput.next();
        loggedIn = false;
        // if user name is correct
                if (USERNAME_ACTUAL.equals(inputedUsername))
                {
                // if both user name and password are correct
                        if (PASSWORD_ACTUAL.equals(inputedPassword))
                        {
                        // LOGGEDIN IS NOT ASSIGNED HERE, this outcome
                        //allows the user
```

This example holds an if else statement, inside of another if else statement, inside of a for loop, inside of a do while loop.

What this does is:

1. It does the things within the do-while as long as the while condition is met

```
                        // to pass the while loop
                        System.out.print("Login successful!");
                        }
                        // if ONLY the USER NAME is correct
                        else
                        {
                        System.out.println("Incorrect password");
                        // if the password is incorrect, logged in is false
                        }
                }
                // if user name is incorrect
                else
                {
                        // if ONLY the password is correct
                        if (PASSWORD_ACTUAL.equals(inputedPassword))
                        {
                        System.out.println("Incorrect username");
                        }
                        // if BOTH are incorrect
                        else
                        {
                        System.out.println("Both user+pass incorrect”.);
                                }
                }
        }// end of for loop
} // end of do
// as long as this is false, it will do
while (loggedIn == true);
```

2. The for loop is allowed to loop a max

Methods

Write/Call

	Return	Name	(Parameter)
static	int	name	int
	void		string
	double		int/double

```
public static void main(String[] args)
        {
                printSummary();
        }

public static void printSummary()
        {

        }
```

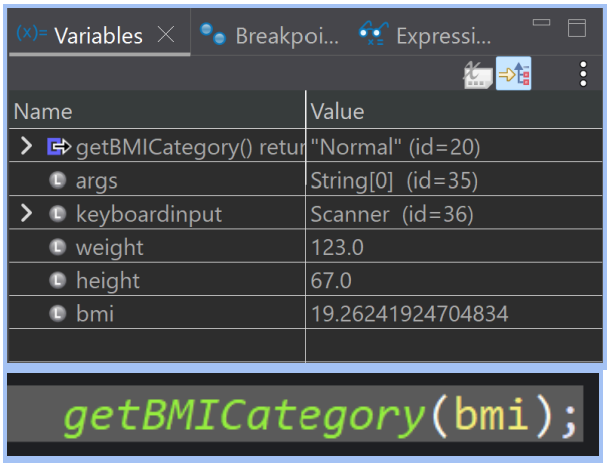
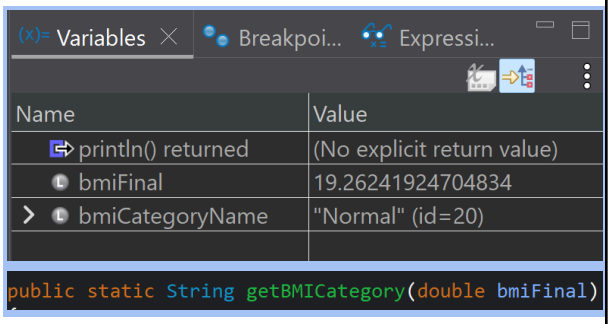
Pass/Returning

Remember to assign variables in other methods after a different one has been called.

```
public static void main(String[] args)
        {
                getPositiveDouble(keyboardinput,
weight);
        }

public static double getPositiveDouble(Scanner input,
double bmiInput)
        {
                bmiInput = input.nextDouble();

                return bmiInput;
        }
```

Trace Memory Methods	
	
Bmi is the value of a double When you want to set the value returned from the method, set whatever variable you want equal to method returned	The double for the new method is defined in the parameters. They both share the same value

Module 4: Arrays

Declaring	Allocating	Initializing
dataType[] arrayReferenceVariable;	arrayReferenceVariable = new dataype[array size (int)]	

Rules:

When an array is created, its size does not change.
All values within the array must be the same data type.

Ref Variables	Memory	Out-Of-Bounds
		When trying to draw an item from the array, when the array isn't that long, it will give an out-of-bounds error.

- Passing arrays by reference to and from methods
- Trace memory during method calls to distinguish reference variables vs. primitive variables, stack vs. heap memory allocation.
- Design algorithms for processing arrays.
- Designing methods that operate on arrays using loops.

Module 5: Classes and Objects

Add the information to demonstrate the following objectives and organize with headings:

- Use classes to define blueprints for creating objects that combine data (fields) and behaviors (methods).
- Define and implement constructors, including default, parameterized, and overloaded constructors, to initialize objects in different ways
- Encapsulation using access modifiers and getters/setters to protect and manage private data.
- Reference variables for objects, stack/heap diagrams.
- Arrays of objects
- Simple file input/output
- Design Implement and Test Java classes that model real-world problems